CyberShield AI - Software Requirements Specification

Version: 1.1
Status: Implemented local MVP specification
Standard: Simplified IEEE 830-inspired structure

1. Introduction

1.1 Purpose

This document specifies the implemented behavior and interfaces of CyberShield AI, a local application for explainable message, URL, and file screening. It is intended for implementation, evaluation, testing, and maintenance.

1.2 Scope

The system provides local account access, three screening methods, per-account scan history and dashboard statistics, model evaluation, and security-awareness content. It does not provide antivirus protection, sandboxing, live threat intelligence, or a guarantee that an input is safe.

1.3 Definitions

Term	Meaning
Ham	Non-spam SMS class in the UCI dataset
Spam	Unwanted or fraudulent SMS class in the UCI dataset
TF-IDF	Term Frequency-Inverse Document Frequency text feature representation
Magic bytes	File-header bytes used to infer a format signature
Risk score	Application-specific numeric score from 0 to 100; not a calibrated probability except where the spam probability is separately identified
Session token	Random bearer token held in an HttpOnly cookie; only its SHA-256 hash is stored server-side

2. System Overview

2.1 Architecture

```
React + TypeScript + Vite browser client
       | JSON / multipart HTTP with HttpOnly session cookie
FastAPI backend
  |-- auth: local accounts, password hash, sessions
  |-- spam: TF-IDF + Logistic Regression pipeline (joblib)
  |-- URL: local heuristic analysis (no network fetch)
  |-- file: byte metadata/signature checks (no execution)
  |-- history/stats: SQLite, scoped by user ID
```

2.2 Runtime environment

- Windows, macOS, or Linux.
- Python 3.11 recommended for the bundled environment and dependencies.
- Node.js 18+ (current frontend toolchain may require a newer Node version; use a supported version for Vite 8).
- Modern browser.
- Network access is only needed for initial dependency installation and optional dataset download; the scanner itself does not call external threat services.

2.3 Constraints and assumptions

- Single local service instance and local SQLite database.
- Bundled model is available for message scanning; retraining requires the UCI dataset file.
- SMS text is expected to be English and at most 2,000 characters.
- URL scanner accepts HTTP/HTTPS URLs up to 2,048 characters and never visits them.
- File scanner limit is 10 MB.
- Local auth is intended for a demonstration/local app, not internet-facing production identity management.

3. Functional Requirements

3.1 Account access

ID	Requirement
AUTH-1	The system shall accept account name (2-80 chars), email (5-254 chars), and password (10-128 chars) at signup.
AUTH-2	The system shall normalize email to lowercase and reject invalid or duplicate email addresses.
AUTH-3	The system shall store only a salted PBKDF2-HMAC-SHA256 password hash and salt, never the submitted password.

ID	Requirement
AUTH-4	The system shall create a random session token, store only its SHA-256 hash, and set a 12-hour HttpOnly, SameSite=Lax cookie.
AUTH-5	The system shall authenticate login with constant-time hash comparison and generic invalid-credential errors.
AUTH-6	The system shall allow the frontend to check the current account and revoke the current session on logout.
AUTH-7	Protected scanner, history, and stats endpoints shall reject missing, invalid, or expired sessions with HTTP 401.

3.2 Message spam scanning

ID	Requirement
MSG-1	Accept message text with length 1-2,000 characters.
MSG-2	Predict `Spam` or `Genuine` using the saved TF-IDF + Logistic Regression model.
MSG-3	Return spam probability, confidence, verdict, and a 0-100 risk score derived from spam probability.
MSG-4	Return up to five positively contributing model terms found in the message.
MSG-5	Return any urgency/link/money-language hints separately from model indicators.
MSG-6	Return safety recommendations and method `ml`.
MSG-7	If the model file is unavailable or invalid, return a clear HTTP 503 response.
MSG-8	The training script shall generate `spam_model.joblib` and `metrics.json` from a local UCI dataset file using a stratified 80/20 split.

3.3 URL analysis

ID	Requirement
URL-1	Accept a URL from 1 to 2,048 characters and reject malformed or unsupported schemes.
URL-2	Check IP host, unusual length, subdomain count, `@`, hyphens, non-HTTPS, sensitive keywords, known shorteners, punycode, selected suspicious TLDs, and known brand names outside expected domains.
URL-3	Add configured rule weights, cap score at 100, and map 0-29 to Low, 30-59 to Medium, and 60-100 to High.

ID	Requirement
URL-4	Return each triggered rule as a human-readable indicator, plus recommendations and method `rule-based`.
URL-5	Never resolve, visit, or fetch the submitted URL.
URL-6	Explicitly label the result “Rule-based analysis, not machine learning”.

3.4 File analysis

ID	Requirement
FILE-1	Accept one uploaded file of at most 10 MB; reject larger files with HTTP 413.
FILE-2	Read the upload as bytes in memory and compute its byte size and SHA-256 hash.
FILE-3	Return the basename, extension, guessed MIME type, recognized magic-byte type, and risk indicators.
FILE-4	Flag selected risky extensions, selected double extensions, and extension/signature mismatches.
FILE-5	Never execute, open, render, or persist the uploaded file bytes.
FILE-6	Return method `metadata` and use a non-assuring low-risk phrase: “No obvious red flags found (not a guarantee)”.

3.5 History and dashboard

ID	Requirement
HIST-1	Store scan user ID, type, truncated input summary, verdict, risk score, optional confidence, method, JSON details, and UTC timestamp.
HIST-2	List at most the latest 200 records for the authenticated user; optionally filter by scan type.
HIST-3	Return a scan detail only when it belongs to the authenticated user; otherwise return HTTP 404.
HIST-4	Clear only the authenticated user's scan history.
DASH-1	Return total scans, high-risk count, counts per scanner type, and latest five scans for the authenticated user.

3.6 Evaluation and awareness

ID	Requirement
EVAL-1	Read and expose accuracy, precision, recall, F1, dataset counts, split, confusion matrix, and limitations from `metrics.json`.
EVAL-2	Return a clear unavailable state if metrics have not been generated.

ID	Requirement
AWARE-1	Present practical security guidance for suspicious messages, links, credentials, files, reporting, and layered protection.

4. External Interfaces

4.1 API endpoints

Method	Endpoint	Access	Purpose
POST	`/api/auth/signup`	Public	Create account and issue session
POST	`/api/auth/login`	Public	Authenticate and issue session
POST	`/api/auth/logout`	Public/current cookie	Revoke current session
GET	`/api/auth/me`	Authenticated	Return current account
GET	`/api/health`	Public	Service and model status
POST	`/api/scan/spam`	Authenticated	Classify message
POST	`/api/scan/url`	Authenticated	Analyze URL without fetching
POST	`/api/scan/file`	Authenticated	Analyze uploaded file metadata
GET	`/api/history`	Authenticated	List own scans; optional `scan_type` filter
GET	`/api/history/{scan_id}`	Authenticated	Get own scan detail
DELETE	`/api/history`	Authenticated	Clear own history
GET	`/api/stats`	Authenticated	Return own dashboard statistics
GET	`/api/model/metrics`	Public	Return model evaluation

4.2 Database tables

`users`: `id`, `name`, case-insensitive unique `email`, `password_hash`, `password_salt`, `created_at`.
`sessions`: hashed token, `user_id`, and `expires_at`.
`scans`: `id`, nullable/migrated `user_id`, scan type, summary, verdict, score, optional confidence, method, JSON details, and timestamp.

5. Non-Functional Requirements

ID	Category	Requirement
NFR-1	Security	Parameterized SQL; salted password hashes; opaque random session tokens; HttpOnly cookie; account-scoped queries; no URL fetch; no file execution
NFR-2	Privacy	Keep scan and account data in local SQLite; persist no uploaded file bytes
NFR-3	Validation	Return clear 4xx responses for invalid input and oversized files; never expose raw stack traces to the UI
NFR-4	Honesty	Do not describe any scanner result as guaranteed safe; disclose the method and limitations
NFR-5	Portability	Python dependencies install from `backend/requirements.txt`; frontend uses `npm install` and Vite scripts
NFR-6	Maintainability	Keep backend routes/analysis/database behavior typed and testable; frontend uses TypeScript
NFR-7	Responsiveness	Core login and application views remain usable on desktop and phone-sized screens

6. Acceptance Tests

The automated pytest suite shall verify:

1. Protected scanner endpoints reject unauthenticated calls.
2. Signup, session verification, logout, and login work.
3. Duplicate accounts and incorrect passwords are rejected.
4. History is isolated across accounts.
5. The documented suspicious URL example reaches High Risk with multiple indicators.
6. A normal HTTPS URL such as `https://www.wikipedia.org` is Low Risk under current rules.
7. Malformed URLs return HTTP 422.
8. File signature mismatch and SHA-256 are reported.
9. A trained spam model returns probability and ML method.
10. Brand-impersonation heuristics are triggered for a known lookalike domain.

The frontend shall type-check and complete `npm run build` successfully. Manual browser checks cover signup, signin, scanner submission, result rendering, and sign-out.

7. Known Limitations

- SMS training/evaluation data is older and English-only; evaluation metrics do not guarantee generalization.

- URL rules are heuristic and may create false positives or miss attacks. Brand-domain recognition is a small explicit list, not a public-suffix-aware reputation service.
- File checks inspect only selected metadata/signatures and cannot detect malicious content.
- Current local cookie is configured for HTTP localhost (`secure=False`); production HTTPS deployment requires secure cookie settings and additional security review.
- Authentication has no email verification, password reset, MFA, rate limiting, or account recovery; it is intended for local demonstration use.
- Existing pre-authentication scan history is not assigned to new accounts by the database migration.
- Dashboard history currently presents a bounded recent list; advanced UI filtering, chart visualizations, and PDF export are not implemented.